05 — Forecasting

Scope. All findings describe the visible subset of pharmacy revenue (counter cash receipts, about 65%); see NB01 for the recording basis. 25 December 2019 (an institutional credit-settlement day, per NB01) is excluded from training. Forecasts cannot anticipate rare large institutional settlements, so actual revenue will occasionally exceed forecasts by a wide margin on such days.

This notebook evaluates SARIMA and Prophet variants against strictly-at-origin baselines using **rolling-origin evaluation**: at every day in the test window, each model forecasts the single day exactly 1, 7 and 30 days ahead (one target day per horizon, not the path in between), using only information available at that origin. SARIMA keeps the order chosen on the training set and re-estimates its parameters on all data up to each origin; every Prophet variant is refit at each origin, so all models are on the same information footing. Baselines are computed strictly from data available at the origin.

Train/test split. Training data: 15 December 2019 to 31 December 2020 (383 calendar days, 347 observed after excluding 25 December 2019 and the missing weeks). Test window: 1 January 2021 to 10 May 2021 (130 calendar days, 122 observed).

Metrics. MASE (headline, scaled by in-sample seasonal-naive MAE on the training set), MAE, and relative MAE against seasonal-naive. All computed on log revenue.

Tests. Every model is tested; no comparison is singled out. At h = 1 and at h = 7, each of the five models (SARIMA and the four Prophet variants) is tested against seasonal-naive with a one-sided Diebold–Mariano test on absolute-error loss (Harvey–Leybourne–Newbold small-sample correction, h – 1 autocovariance lags); the alternative is lower expected absolute error than seasonal-naive. Within each horizon the five p-values are Holm-adjusted and judged at 2.5%, so the chance of any false finding across the two horizons is at most 5%. h = 30 is reported as point estimates only: its 85 forecasts overlap by up to 29 days, leaving about three independent windows, too few for a reliable test. Last-value is not a candidate: at h = 7 it coincides with seasonal-naive. The size of this procedure under a null built from the data's own loss differentials is checked in `validation/dm_calibration.py`.

Alongside the test, each model's relative MAE is reported with a two-sided 95% interval from the same standard error, and the minimum gain detectable with 80% power at the 2.5% one-sided level. These are descriptive; the Holm-adjusted p-value is the test.

In [1]:
```
import warnings
import logging

# Silence tqdm / IProgress warnings from Prophet's dependency chain, then
```

```
# route Prophet + cmdstanpy log output to a null handler so the
# rolling-origin loop does not spam 400+ chain-start / chain-done lines.
warnings.filterwarnings("ignore", message="IProgress not found")
warnings.filterwarnings("ignore", message="Could not infer format")
warnings.filterwarnings(
    "ignore", message="Optimization terminated abnormally"
)
for _name in ("cmdstanpy", "prophet"):
    _lg = logging.getLogger(_name)
    _lg.handlers = [logging.NullHandler()]
    _lg.propagate = False
    _lg.setLevel(logging.ERROR)

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from pathlib import Path
from scipy import stats
import statsmodels.api as sm
from statsmodels.tsa.statespace.sarimax import SARIMAX
import ruptures as rpt
from prophet import Prophet

sns.set_theme(style="whitegrid", context="notebook")
plt.rcParams["figure.figsize"] = (11, 4)
plt.rcParams["axes.titleweight"] = "bold"
```

In [2]:

```
df = pd.read_csv("../data/daily_sales_final.csv", parse_dates=["date"])
df = df.sort_values("date").set_index("date").asfreq("D")
df.loc[pd.Timestamp("2019-12-25"), "revenue_scaled"] = np.nan

# Daily-frequency log series with NaN for missing days: SARIMAX handles NaN
# via the Kalman filter, which keeps calendar (weekday) alignment intact
# through missing days and weeks.
y_raw = df["revenue_scaled"]
log_y_daily = np.log(y_raw)
log_y_observed = log_y_daily.dropna()

TRAIN_END = pd.Timestamp("2020-12-31")
TEST_START = pd.Timestamp("2021-01-01")
TEST_END = log_y_observed.index.max()

train_daily = log_y_daily.loc[:TRAIN_END]  # for SARIMA (has NaN)
train_observed = log_y_observed.loc[
    :TRAIN_END
]  # for Prophet (observed only)
test = log_y_observed.loc[TEST_START:TEST_END]

train_start = TRAIN_END - pd.Timedelta(train_daily.shape[0] - 1, "D")
print(f"Training window: {train_start.date()} to {TRAIN_END.date()}")
print(
    f"  Calendar days: {len(train_daily)}, "
    f"observed days: {len(train_observed)}"
)
print(
```

```
    f"Test window:     {test.index.min().date()} "
    f"to {test.index.max().date()}"
)
print(
    f"  Calendar days: {(TEST_END - TEST_START).days + 1}, "
    f"observed days: {len(test)}"
)
```

```
Training window: 2019-12-15 to 2020-12-31
  Calendar days: 383, observed days: 347
Test window:     2021-01-01 to 2021-05-10
  Calendar days: 130, observed days: 122
```

SARIMA order: brief AIC comparison

Fit a small grid of SARIMA(p,d,q)(P,D,Q,7) orders on the training set and pick the lowest-AIC combination whose ML optimisation converged. Non-convergent fits have untrustworthy AIC and are excluded.

In [3]:
```
grid_rows = []
with warnings.catch_warnings():
    warnings.simplefilter("ignore")
    for p in [0, 1, 2]:
        for q in [0, 1, 2]:
            for P in [0, 1]:
                for Q in [0, 1]:
                    order = (p, 1, q)
                    seasonal_order = (P, 1, Q, 7)
                    try:
                        res = SARIMAX(
                            train_daily.values,
                            order=order,
                            seasonal_order=seasonal_order,
                        ).fit(disp=False)
                        grid_rows.append(
                            {
                                "order": order,
                                "seasonal_order": seasonal_order,
                                "AIC": res.aic,
                                "converged": bool(
                                    res.mle_retvals.get("converged", True)
                                ),
                            }
                        )
                    except Exception:
                        continue

grid_df = pd.DataFrame(grid_rows).sort_values("AIC").reset_index(drop=True)
converged_top = grid_df[grid_df["converged"]].head(10)
with pd.option_context("display.float_format", "{:.2f}".format):
    print("Top 10 SARIMA orders by AIC (converged only):")
    print(converged_top.to_string(index=False))

best = converged_top.iloc[0]
```

```
CHOSEN_ORDER = best["order"]
CHOSEN_SEASONAL = best["seasonal_order"]
n_nonconv = int((~grid_df["converged"]).sum())
print()
print(f"Non-convergent fits excluded: {n_nonconv} of {len(grid_df)}")
print(
    "Chosen (lowest AIC among converged): "
    f"SARIMA{CHOSEN_ORDER}{CHOSEN_SEASONAL}"
)
```

```
Top 10 SARIMA orders by AIC (converged only):
    order seasonal_order     AIC  converged
(0, 1, 1)   (0, 1, 1, 7) -358.87       True
(0, 1, 1)   (1, 1, 1, 7) -358.60       True
(1, 1, 2)   (1, 1, 1, 7) -358.08       True
(1, 1, 2)   (0, 1, 1, 7) -357.76       True
(1, 1, 1)   (0, 1, 1, 7) -357.00       True
(0, 1, 2)   (0, 1, 1, 7) -357.00       True
(1, 1, 1)   (1, 1, 1, 7) -356.66       True
(0, 1, 2)   (1, 1, 1, 7) -356.66       True
(2, 1, 2)   (0, 1, 1, 7) -355.88       True
(2, 1, 1)   (0, 1, 1, 7) -355.08       True

Non-convergent fits excluded: 1 of 36
Chosen (lowest AIC among converged): SARIMA(0, 1, 1)(0, 1, 1, 7)
```

Training-only PELT for Prophet variant B

The full-sample NB02 break search identifies breaks at 24 March, 28 May and 4 December 2020. A forecaster at 31 December 2020 does not have data past their training window, so using the December break as a Prophet changepoint would be look-ahead. This cell reruns PELT on the training data only and uses whatever it detects as the changepoint set for Prophet variant B.

In [4]:
```
train_log = train_observed.dropna()
train_dow_means = train_log.groupby(train_log.index.dayofweek).transform(
    "mean"
)
train_signal = (train_log - train_dow_means).values

algo = rpt.Pelt(model="rbf", jump=1, min_size=30).fit(train_signal)
detected = algo.predict(pen=10)[:-1]
training_breaks = [train_log.index[i].date() for i in detected]
print(f"Training-only PELT breaks (pen=10): {training_breaks}")
```

```
Training-only PELT breaks (pen=10): [datetime.date(2020, 3, 24), datetime.da
te(2020, 5, 28)]
```

Rolling-origin evaluation

At each day `t` in the test window, forecast log revenue at `t + h` for `h ∈ {1, 7, 30}` using only information available at `t`:

- **SARIMA** (chosen order): re-estimate the parameters on the daily-frequency series up to each origin (`sarima_train.apply(new_series, refit=True)` , warm-started from the previous origin's estimates; if the optimiser fails, the previous estimates are kept and the failure counted), then forecast up to 30 days ahead. Missing calendar days in the series are handled by the Kalman filter, which preserves weekday alignment through missing days and weeks.
- **Prophet default, variants A, B, C**: refit at every origin. See the four `Prophet(...)` fits below.
- **Seasonal-naive**: same weekday from the most recent same-weekday observation at or before `t` .
- **Last-value**: last observed value at or before `t` .

Runtime: 4 Prophet variants × 122 origins = 488 Prophet fits, plus 122 SARIMA fits. The full notebook runs in one to two minutes on a modern laptop.

In [5]:
```
HORIZONS = [1, 7, 30]


def seasonal_naive_forecast(series_up_to_origin, forecast_date):
    """Same weekday from the most recent same-weekday observation."""
    target_dow = forecast_date.dayofweek
    same_dow = series_up_to_origin[
        series_up_to_origin.index.dayofweek == target_dow
    ]
    if len(same_dow) == 0:
        return series_up_to_origin.dropna().iloc[-1]
    return same_dow.dropna().iloc[-1]


def last_value_forecast(series_up_to_origin):
    return series_up_to_origin.dropna().iloc[-1]


def prophet_fit(train_series, changepoints=None, post_covid_split=None):
    d = pd.DataFrame(
        {"ds": train_series.index, "y": train_series.values}
    ).dropna(subset=["y"])
    kwargs = dict(
        weekly_seasonality=False if post_covid_split is not None else True,
        yearly_seasonality=False,
        daily_seasonality=False,
    )
    if changepoints is not None:
        kwargs["changepoints"] = [str(c) for c in changepoints]
    if post_covid_split is not None:
        d["post_covid"] = (
            d["ds"] >= pd.Timestamp(post_covid_split)
        ).astype(int)
        d["pre_covid"] = 1 - d["post_covid"]
    m = Prophet(**kwargs)
    if post_covid_split is not None:
```

```
        m.add_seasonality(
            name="weekly_pre",
            period=7,
            fourier_order=3,
            condition_name="pre_covid",
        )
        m.add_seasonality(
            name="weekly_post",
            period=7,
            fourier_order=3,
            condition_name="post_covid",
        )
    m.fit(d)
    return m


def prophet_predict(
    model, last_train_date, forecast_dates, post_covid_split=None
):
    horizon = int((max(forecast_dates) - last_train_date).days)
    future = model.make_future_dataframe(periods=horizon, freq="D")
    if post_covid_split is not None:
        future["post_covid"] = (
            future["ds"] >= pd.Timestamp(post_covid_split)
        ).astype(int)
        future["pre_covid"] = 1 - future["post_covid"]
    pred = model.predict(future).set_index("ds")["yhat"]
    return pred.loc[forecast_dates]


# Fit SARIMA once on training (daily-frequency series with NaN). Each
# origin re-estimates the parameters, warm-started from the previous
# origin's estimates to save iterations.
sarima_train = SARIMAX(
    train_daily.values,
    order=CHOSEN_ORDER,
    seasonal_order=CHOSEN_SEASONAL,
).fit(disp=False)
sarima_params = sarima_train.params
sarima_nonconverged = 0

test_origins = test.index

# The prophet_predict call uses the last-training-day of the Prophet fit
# for its make_future_dataframe start; when we refit daily, that is always
# the origin.
records = []
observed_dates = set(log_y_daily.dropna().index)

for origin in test_origins:
    combined_daily = log_y_daily.loc[:origin]
    combined_observed = log_y_observed.loc[:origin]
    last_obs_date = origin

    # SARIMA: same order, parameters re-estimated on data up to the origin.
    # If the optimiser fails, keep the last converged parameters for this
```

```
    # origin and count it.
    with warnings.catch_warnings():
        warnings.simplefilter("ignore")
        sarima_res_here = sarima_train.apply(
            combined_daily.values,
            refit=True,
            fit_kwargs={"start_params": sarima_params, "disp": False},
        )
    if sarima_res_here.mle_retvals.get("converged", True):
        sarima_params = sarima_res_here.params
    else:
        sarima_nonconverged += 1
        sarima_res_here = sarima_res_here.model.filter(sarima_params)
    sarima_fc_vals = sarima_res_here.forecast(steps=max(HORIZONS))
    sarima_fc_index = pd.date_range(
        origin + pd.Timedelta(1, "D"), periods=max(HORIZONS), freq="D"
    )
    sarima_fc = pd.Series(sarima_fc_vals, index=sarima_fc_index)

    prophet_models = {
        "default": prophet_fit(combined_observed),
        "variant_A": prophet_fit(
            combined_observed, changepoints=["2020-03-24"]
        ),
        "variant_B": prophet_fit(
            combined_observed,
            changepoints=[str(d) for d in training_breaks],
        ),
        "variant_C": prophet_fit(
            combined_observed, post_covid_split="2020-03-24"
        ),
    }

    for h in HORIZONS:
        forecast_date = origin + pd.Timedelta(h, "D")
        if forecast_date > TEST_END:
            continue
        if forecast_date not in observed_dates:
            continue
        actual = log_y_daily.loc[forecast_date]
        if pd.isna(actual):
            continue

        row = {
            "origin": origin,
            "horizon": h,
            "forecast_date": forecast_date,
            "actual": actual,
            "sarima": sarima_fc.loc[forecast_date]
            if forecast_date in sarima_fc.index
            else np.nan,
            "seasonal_naive": seasonal_naive_forecast(
                combined_observed, forecast_date
            ),
            "last_value": last_value_forecast(combined_observed),
        }
```

```
        for variant in ["default", "variant_A", "variant_B", "variant_C"]:
            pred = prophet_predict(
                prophet_models[variant],
                last_obs_date,
                [forecast_date],
                post_covid_split="2020-03-24"
                if variant == "variant_C"
                else None,
            )
            row[f"prophet_{variant}"] = float(pred.iloc[0])
        records.append(row)

results = pd.DataFrame(records)
print(
    f"Rolling-origin loop complete: {len(records)} forecast rows "
    f"across {len(test_origins)} origins."
)
print("Prophet refits: 122 origins × 4 variants = 488 fits.")
print(
    f"SARIMA refits: {len(test_origins)} (non-converged, previous "
    f"parameters kept: {sarima_nonconverged})."
)
```

```
Rolling-origin loop complete: 311 forecast rows across 122 origins.
Prophet refits: 122 origins × 4 variants = 488 fits.
SARIMA refits: 122 (non-converged, previous parameters kept: 3).
```

Metrics: MASE, MAE, relative MAE against seasonal-naive

MASE is scaled by the in-sample seasonal-naive MAE on the training set. All quantities are on log revenue.

In [6]:

```
NAME_MAP = {
    "seasonal_naive": "Seasonal-naive",
    "last_value": "Last-value",
    "sarima": "SARIMA",
    "prophet_default": "Prophet (default)",
    "prophet_variant_A": "Prophet (variant A)",
    "prophet_variant_B": "Prophet (variant B)",
    "prophet_variant_C": "Prophet (variant C)",
}
MODELS = list(NAME_MAP.keys())

# Use the daily-frequency series so the lag-7 difference compares
# calendar-equivalent weekdays (a gap-free series with a single missing day
# would shift one week of comparisons by one weekday).
train_seasonal_naive_err = (
    (train_daily - train_daily.shift(7)).abs().dropna()
)
train_sn_mae = float(train_seasonal_naive_err.mean())
print(f"In-sample seasonal-naive MAE (log units): {train_sn_mae:.4f}")
print()
```

```
metrics_rows = []
for h in HORIZONS:
    sub = results[results["horizon"] == h].copy()
    sn_mae_h = float((sub["seasonal_naive"] - sub["actual"]).abs().mean())
    for model in MODELS:
        errs = (sub[model] - sub["actual"]).abs()
        mae = float(errs.mean())
        mase = mae / train_sn_mae
        rel_mae_sn = mae / sn_mae_h
        metrics_rows.append(
            {
                "model": NAME_MAP[model],
                "h": h,
                "n": int(len(sub)),
                "MAE (log)": mae,
                "MASE": mase,
                "MAE / seasonal-naive MAE": rel_mae_sn,
            }
        )

metrics_df = pd.DataFrame(metrics_rows)
with pd.option_context("display.float_format", "{:.4f}".format):
    print("Rolling-origin metrics:")
    print(metrics_df.to_string(index=False))
```

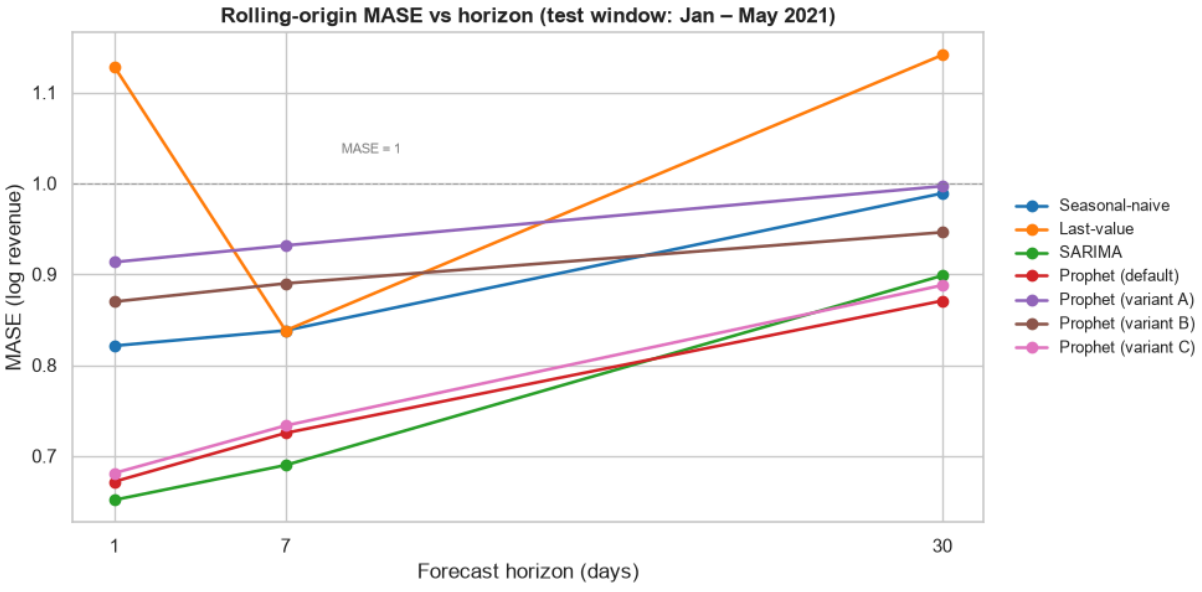
```
In-sample seasonal-naive MAE (log units): 0.1541

Rolling-origin metrics:
              model  h   n  MAE (log)   MASE  MAE / seasonal-naive MAE
     Seasonal-naive  1 119     0.1266 0.8212                    1.0000
         Last-value  1 119     0.1739 1.1284                    1.3740
             SARIMA  1 119     0.1004 0.6516                    0.7934
  Prophet (default)  1 119     0.1035 0.6717                    0.8180
Prophet (variant A)  1 119     0.1408 0.9133                    1.1121
Prophet (variant B)  1 119     0.1341 0.8698                    1.0591
Prophet (variant C)  1 119     0.1050 0.6811                    0.8293
     Seasonal-naive  7 107     0.1292 0.8380                    1.0000
         Last-value  7 107     0.1292 0.8380                    1.0000
             SARIMA  7 107     0.1063 0.6899                    0.8232
  Prophet (default)  7 107     0.1118 0.7252                    0.8654
Prophet (variant A)  7 107     0.1436 0.9316                    1.1117
Prophet (variant B)  7 107     0.1371 0.8898                    1.0618
Prophet (variant C)  7 107     0.1130 0.7334                    0.8752
     Seasonal-naive 30  85     0.1524 0.9890                    1.0000
         Last-value 30  85     0.1759 1.1411                    1.1538
             SARIMA 30  85     0.1385 0.8985                    0.9085
  Prophet (default) 30  85     0.1342 0.8707                    0.8805
Prophet (variant A) 30  85     0.1536 0.9968                    1.0080
Prophet (variant B) 30  85     0.1458 0.9460                    0.9566
Prophet (variant C) 30  85     0.1368 0.8878                    0.8977
```

MASE vs horizon

In [7]:
```
fig, ax = plt.subplots(figsize=(10, 5))
palette = sns.color_palette("tab10", n_colors=len(MODELS))
```

```
for m, color in zip(MODELS, palette):
    d = metrics_df[metrics_df["model"] == NAME_MAP[m]].sort_values("h")
    ax.plot(
        d["h"],
        d["MASE"],
        marker="o",
        label=NAME_MAP[m],
        color=color,
        linewidth=1.8,
    )
ax.axhline(1.0, color="grey", linestyle="--", linewidth=0.8, alpha=0.6)
ax.text(
    10,
    1.03,
    "MASE = 1",
    fontsize=8,
    color="grey",
    va="bottom",
    ha="center",
)
ax.set_xlabel("Forecast horizon (days)")
ax.set_ylabel("MASE (log revenue)")
ax.set_title(
    "Rolling-origin MASE vs horizon (test window: Jan – May 2021)"
)
ax.set_xticks(HORIZONS)
ax.legend(
    loc="center left",
    bbox_to_anchor=(1.02, 0.5),
    fontsize=9,
    frameon=False,
)
plt.tight_layout()
plt.savefig(
    "../charts/nb05_mase_vs_horizon.png", dpi=150, bbox_inches="tight"
)
plt.show()
```



Test against seasonal-naive: one-sided DM with Holm at h = 1 and h = 7

Loss is absolute error on log revenue. For each model, d_t = |e_SN,t| − |e_model,t| (positive when the model is better), and the DM statistic is d̄ over its HLN-corrected standard error, with h − 1 autocovariance lags because h-step-ahead errors are at most MA(h − 1). The one-sided p-value is P(t_{n−1} > DM). Within each horizon the five p-values are Holm-adjusted; a model beats the benchmark if its adjusted p-value is below 2.5%. The rectangular-kernel variance is not guaranteed positive; if it fails, Bartlett weights at the same bandwidth are used instead, and the cell prints whether this happened. The descriptive columns give each model's relative MAE, its two-sided 95% interval, and the minimum detectable gain, 2.80 × SE (one-sided 2.5%, 80% power), relative to seasonal-naive MAE with the denominator treated as fixed. The forecast table is saved to `validation/nb05_rolling_forecasts.csv` for the calibration script.

In [8]:

```
from statsmodels.stats.multitest import multipletests

from pharmacy_covid_analysis.forecast_tests import dm_test_hln

CANDIDATES = [
    "sarima",
    "prophet_default",
    "prophet_variant_A",
    "prophet_variant_B",
    "prophet_variant_C",
]
TEST_HORIZONS = [1, 7]
ALPHA_ONE_SIDED = 0.025

results.to_csv("../validation/nb05_rolling_forecasts.csv", index=False)


# 2.80 x SE: one-sided 2.5% critical value plus the 80%-power quantile
MDE_MULTIPLIER = stats.norm.ppf(1 - ALPHA_ONE_SIDED) + stats.norm.ppf(0.80)

rows = []
n_fallback = 0
for h in TEST_HORIZONS:
    sub = results[results["horizon"] == h]
    e_sn = (sub["actual"] - sub["seasonal_naive"]).to_numpy()
    mae_sn = float(np.abs(e_sn).mean())
    t_crit = stats.t.ppf(0.975, df=len(sub) - 1)
    pvals = []
    for c in CANDIDATES:
        e_m = (sub["actual"] - sub[c]).to_numpy()
        stat, p, dbar, se, fallback = dm_test_hln(e_m, e_sn, h=h)
        n_fallback += fallback
        pvals.append(p)
        rows.append(
            {
```

```
                "h": h,
                "model": NAME_MAP[c],
                "n": len(sub),
                "MAE vs SN (%)": -dbar / mae_sn * 100,
                "95% low (%)": -(dbar + t_crit * se) / mae_sn * 100,
                "95% high (%)": -(dbar - t_crit * se) / mae_sn * 100,
                "DM": stat,
                "p (one-sided)": p,
                "min. detectable gain (%)": MDE_MULTIPLIER
                * se
                / mae_sn
                * 100,
            }
        )
    reject, p_holm, _, _ = multipletests(
        pvals, alpha=ALPHA_ONE_SIDED, method="holm"
    )
    for row, p_adj, rej in zip(rows[-len(CANDIDATES) :], p_holm, reject):
        row["p (Holm)"] = p_adj
        row["beats SN at 2.5%"] = "yes" if rej else "no"

test_df = pd.DataFrame(rows)
print("One-sided DM tests against seasonal-naive, Holm-adjusted within")
print(
    "each horizon (negative MAE vs SN = lower error than seasonal-naive):"
)
with pd.option_context("display.float_format", "{:.4f}".format):
    for h in TEST_HORIZONS:
        print()
        print(
            test_df[test_df["h"] == h]
            .drop(columns="h")
            .to_string(index=False)
        )
print()
print(f"Bartlett fallback used: {n_fallback} of {len(rows)} tests.")
```

```
One-sided DM tests against seasonal-naive, Holm-adjusted within
each horizon (negative MAE vs SN = lower error than seasonal-naive):

              model   n  MAE vs SN (%)  95% low (%)  95% high (%)      DM  p
(one-sided)  min. detectable gain (%)  p (Holm) beats SN at 2.5%
             SARIMA 119       -20.6573     -34.4183       -6.8963  2.9727
0.0018                    19.4684    0.0089               yes
  Prophet (default) 119       -18.2039     -33.4056       -3.0021  2.3713
0.0097                    21.5066    0.0387                no
Prophet (variant A) 119        11.2092      -6.8563       29.2747 -1.2287
0.8892                    25.5581    1.0000                no
Prophet (variant B) 119         5.9106     -11.8825       23.7037 -0.6578
0.7440                    25.1728    1.0000                no
Prophet (variant C) 119       -17.0650     -32.5366       -1.5935  2.1842
0.0155                    21.8883    0.0464                no

              model   n  MAE vs SN (%)  95% low (%)  95% high (%)      DM  p
(one-sided)  min. detectable gain (%)  p (Holm) beats SN at 2.5%
             SARIMA 107       -17.6760     -27.3501       -8.0019  3.6225
0.0002                    13.6704    0.0011               yes
  Prophet (default) 107       -13.4593     -34.3742        7.4556  1.2759
0.1024                    29.5546    0.4096                no
Prophet (variant A) 107        11.1689     -19.4973       41.8352 -0.7221
0.7641                    43.3341    1.0000                no
Prophet (variant B) 107         6.1820     -22.6732       35.0372 -0.4248
0.6641                    40.7749    1.0000                no
Prophet (variant C) 107       -12.4817     -32.1834        7.2199  1.2560
0.1059                    27.8402    0.4096                no

Bartlett fallback used: 0 of 10 tests.
```

Error by weekday at h = 7 — link to NB04

NB04 finds that the Friday depression deepened across regimes. Prophet default fits one weekly pattern pooled across all training regimes; Prophet variant C splits the weekly pattern at 24 March. One hypothesis before running the check: Prophet default might be under-weighting the post-COVID Friday dip in its pooled fit, so variant C's regime-split should reduce Friday error. The table below breaks h = 7 absolute errors by weekday for SARIMA, Prophet default, Prophet variant C and the seasonal-naive baseline. The breakdown is descriptive; no weekday comparison is tested. The cell after it prints the fitted Friday effect of Prophet default and variant C against the plateau's empirical Friday deviation (SARIMA has no fitted weekly component to read off).

In [9]:
```python
weekday_names = {
    0: "Mon",
    1: "Tue",
    2: "Wed",
    3: "Thu",
    4: "Fri",
    5: "Sat",
    6: "Sun",
}
```

```
sub_h7 = results[results["horizon"] == 7].copy()
sub_h7["weekday"] = sub_h7["forecast_date"].dt.dayofweek.map(weekday_names)
by_wd_rows = []
for wd in ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"]:
    s = sub_h7[sub_h7["weekday"] == wd]
    if len(s) == 0:
        continue
    row = {"weekday": wd, "n": int(len(s))}
    for model in [
        "sarima",
        "prophet_default",
        "prophet_variant_C",
        "seasonal_naive",
    ]:
        row[NAME_MAP[model]] = float(np.abs(s[model] - s["actual"]).mean())
    by_wd_rows.append(row)
by_wd = pd.DataFrame(by_wd_rows)
with pd.option_context("display.float_format", "{:.4f}".format):
    print("Absolute forecast error by weekday at h = 7 (log units):")
    print(by_wd.to_string(index=False))
```

```
Absolute forecast error by weekday at h = 7 (log units):
weekday  n  SARIMA  Prophet (default)  Prophet (variant C)  Seasonal-naive
    Mon 16  0.1496            0.1525              0.1513          0.1867
    Tue 13  0.1147            0.1136              0.1154          0.1525
    Wed 15  0.0918            0.0944              0.0954          0.1308
    Thu 15  0.0853            0.1054              0.1031          0.1131
    Fri 16  0.1142            0.1318              0.1416          0.1134
    Sat 16  0.1042            0.1027              0.1010          0.0989
    Sun 16  0.0838            0.0808              0.0821          0.1121
```

In [10]:

```
# Fitted Friday effect of Prophet default and variant C at an early
# test-window origin (2021-01-31), against the plateau's empirical Friday
# deviation.
diag_train_end = pd.Timestamp("2021-01-31")
diag_train = log_y_observed.loc[:diag_train_end]

m_diag_def = prophet_fit(diag_train)
m_diag_C = prophet_fit(diag_train, post_covid_split="2020-03-24")

diag_dates = pd.date_range("2021-01-31", periods=7, freq="D")
pred_def = m_diag_def.predict(pd.DataFrame({"ds": diag_dates}))
d_C_future = pd.DataFrame({"ds": diag_dates})
d_C_future["post_covid"] = 1
d_C_future["pre_covid"] = 0
pred_C = m_diag_C.predict(d_C_future)

friday_row = diag_dates.dayofweek.tolist().index(4)  # Friday
default_fri_effect = float(pred_def.iloc[friday_row]["weekly"])
variantC_fri_effect = float(pred_C.iloc[friday_row]["weekly_post"])

plateau = log_y_observed.loc["2020-12-04":"2021-05-10"]
plateau_fri = float(
    plateau[plateau.index.dayofweek == 4].mean() - plateau.mean()
)
```

```
print("Fitted Friday effect (log units, at training end 2021-01-31):")
print(
    f"  Prophet default:                       {default_fri_effect:+.4f}"
)
print(
    f"  Prophet variant C (post-COVID pattern): {variantC_fri_effect:+.4f}"
)
print(f"  Plateau's empirical Friday deviation:   {plateau_fri:+.4f}")
```

```
Fitted Friday effect (log units, at training end 2021-01-31):
  Prophet default:                       -0.2406
  Prophet variant C (post-COVID pattern): -0.2785
  Plateau's empirical Friday deviation:   -0.2081
```

Reading the Friday-effect arithmetic. The hypothesis above is wrong in direction: both Prophet variants *overshoot* the plateau's Friday dip, and variant C overshoots more. Default pools across all four regimes in the data to 31 January 2021; the crash and rebound Fridays are deeper (about −0.26 to −0.32 as deviations from the weekly mean) than the plateau's own (about −0.21), which drags default's pooled Friday effect below the plateau's actual. (All figures here are deviations from the weekly mean. NB04 reports Friday at −26% to −31% against the other six days, which is −0.30 to −0.37 in logs and −0.26 to −0.32 on this scale, a factor of 6/7.) Variant C's post-COVID pattern *excludes* the pre-COVID Fridays (which were the shallowest, about −0.12) and pools only crash, rebound and plateau, so variant C's post-COVID Friday effect is deeper still than default's pool. This is why variant C, designed to help with the Friday-by-regime finding from NB04, makes Friday error worse in this test window: it removes the shallowest Fridays that were the only ones pulling default's pool towards the plateau's own. With only 16 Friday forecasts at h = 7 some residual noise is expected, but the arithmetic here matches the direction of the observed error.

Reading the results

The MASE-vs-horizon plot and the metrics table are the main outputs. The tests are the one-sided DM tests above; comparisons at h = 30 and between models are described as point estimates rather than "wins". Seasonal-naive's own MASE is below 1 (0.82 at h = 1) because the scaling denominator is the training-window seasonal-naive error, which includes the crash and rebound; MASE compares horizons, and the relative-MAE column compares models with the rule. Three of the 122 SARIMA refits did not converge and kept the previous origin's estimates.

By horizon.

- **h = 1** (119 forecasts): Point estimates favour SARIMA (MASE 0.65), Prophet default (0.67) and variant C (0.68) : a three-way near-tie. Seasonal-naive is at 0.82, last-value at 1.13. Last-value is worse than seasonal-naive at h = 1 because the last observed value can be any weekday, while seasonal-naive matches weekdays.

- **h = 7** (107 forecasts): Point estimates favour SARIMA (0.69), then Prophet default (0.73) and variant C (0.73). At h = 7 the target has the origin's weekday, so the two baselines coincide by construction: last-value uses the origin's value, and seasonal-naive uses the most recent same-weekday value, which is the origin itself.
- **h = 30** (85 forecasts): Point estimates favour Prophet default (0.87), variant C (0.89), SARIMA (0.90); the top three are within 0.03 of each other. Seasonal-naive at 0.99.

Tests against seasonal-naive. At both horizons SARIMA is the only model that beats seasonal-naive after Holm adjustment: one-sided p = 0.0018 at h = 1 (Holm-adjusted 0.0089) and 0.0002 at h = 7 (adjusted 0.0011), with MAE 20.7% and 17.7% below the rule (95% intervals 6.9–34.4% and 8.0–27.4% lower). Prophet default's lower error (18.2% and 13.5%) does not survive adjustment (adjusted p = 0.039 and 0.41); variant C is in the same position; variants A and B are worse than the rule in point terms. The verdict does not depend on the choice of comparisons: SARIMA's one-sided p-values are below 0.005, the Bonferroni threshold across the ten tests run, and below 0.0033 even counting the five untested h = 30 comparisons; the margin is wide at h = 7 and narrow at h = 1. The Bartlett fallback was not needed. Under a null built from the data's own loss differentials (figures from `validation/dm_calibration.py`, run separately: 1,000 null draws per horizon by stationary bootstrap with 7-day blocks, Monte Carlo standard errors of 0.4–0.8 points), the Holm procedure's family-wise error rate at nominal 2.5% is 1.9% at h = 1 and 5.9% at h = 7; SARIMA's adjusted p at h = 7 (0.0011) sits far below 2.5%, so the over-rejection there cannot change the verdict. `arch`'s SPA test, which does not studentise (in arch 8.0.0 the `studentize` flag only sets the recentring threshold; the statistic is the raw maximum of the mean loss differentials, White's statistic with Hansen's recentring), does not reject on the same forecasts at 2.5% (consistent p = 0.044 at h = 1 and 0.117 at h = 7). The two variants that are worse than the rule, whose differentials are the noisiest, dominate its bootstrap maximum: SARIMA alone gives p ≤ 0.001 at both horizons, and adding either of those variants raises it to 0.03–0.10 (`validation/dm_calibration.py`). Hansen's studentised SPA, which removes that weakness (`pharmacy_covid_analysis/spa.py`), rejects at both horizons (p = 0.010 and 0.006; StepM names SARIMA only). Under the data-based null, on the same draws, the rejection rates at nominal 2.5% are 1.9% and 5.9% for the Holm procedure, 4.4% and 6.7% for `arch`'s SPA and 3.9% and 5.7% for the studentised SPA: the Holm procedure is better calibrated a day ahead (1.9% against 3.9–4.4%, a difference well beyond simulation error) and comparable a week ahead, and it needs no block length or seed, so it is the test used; the studentised SPA agrees with it. Minimum detectable gains are 14–30% for the three leading models (SARIMA, Prophet default and variant C), and up to 43% for variants A and B: the sample can establish large gains only, which is why Prophet's 13.5% at h = 7 is neither confirmed nor ruled out. No model carries a Ramadan term; Ramadan 2021 (13 April to 12 May) lies inside the test window, and NB04 estimates its effect at about −3% (not significant), so its influence on the h = 30 forecasts into late April and May is small but unmodelled.

By weekday. SARIMA beats seasonal-naive on Sunday to Thursday by 20% (Monday) to 30% (Wednesday), is level on Friday (1% worse) and 5% worse on Saturday. Prophet default beats seasonal-naive on Sunday to Thursday by 7% (Thursday) to 28% (Wednesday and Sunday), but is 16% worse on Friday and 4% worse on Saturday: it loses on both days of the local weekend. Variant C, designed to help on Fridays, is worse still (25% worse on Friday, 2% worse on Saturday). The gap between the two models is concentrated on Thursday and Friday. A likely reason: SARIMA updates its weekly pattern from recent weeks, much like additive Holt–Winters smoothing, so on the plateau it tracks the current Friday dip; Prophet fits one weekly pattern to all the data since December 2019. The cell above prints the fitted Friday effects of Prophet default and variant C against the plateau's empirical deviation.

Prophet variants A and B. Both underperform Prophet default at every horizon.

- Variant A (single manual changepoint at 24 March 2020) is worse than Prophet default's automatic changepoint detection on this training window.
- Variant B (training-only PELT breaks). The training data alone reveal only the March and May breaks; the 4 December break is not in the training-only signal because only 21 observed days (28 calendar days) of the plateau precede the cutoff, fewer than PELT's minimum segment length of 30 observed days (the NB02 setting), so the December break cannot be placed by construction. Using NB02's full-sample dates as Prophet changepoints would be look-ahead; that is the methodological point of the variant B design. The PELT breakset here is also fixed at 31 December 2020; a real-time forecaster would rerun the detection as 2021 data arrived and would pick up the 4 December break some weeks into the test window.

Secondary result: single-shot forecast from 31 December 2020

Fit each model once on the training window and forecast all 130 days of the test window in one shot. Every model uses training data only. This is the forecast a planner would run once at the start of the year; it is reported alongside the rolling-origin evaluation.

In [11]:
```
train_only_sarima = SARIMAX(
    train_daily.values,
    order=CHOSEN_ORDER,
    seasonal_order=CHOSEN_SEASONAL,
).fit(disp=False)
test_calendar_days = int((TEST_END - TRAIN_END).days)
sarima_full_fc = pd.Series(
    train_only_sarima.forecast(steps=test_calendar_days),
    index=pd.date_range(
        TRAIN_END + pd.Timedelta(1, "D"),
        periods=test_calendar_days,
        freq="D",
    ),
```

```
).loc[test.index]

prophet_full = prophet_fit(train_observed)
future = prophet_full.make_future_dataframe(
    periods=test_calendar_days, freq="D"
)
prophet_full_fc = (
    prophet_full.predict(future).set_index("ds")["yhat"].loc[test.index]
)


def sn_fixed_origin(fc_date, train_series):
    same_dow = train_series[
        train_series.index.dayofweek == fc_date.dayofweek
    ]
    return same_dow.dropna().iloc[-1]


sn_full_fc = pd.Series(
    [sn_fixed_origin(d, train_observed) for d in test.index],
    index=test.index,
)


def eval_fixed(fc, actual):
    err = (fc - actual).abs().dropna()
    return float(err.mean())


fixed_rows = []
for name, fc in [
    ("SARIMA (single-shot)", sarima_full_fc),
    ("Prophet default (single-shot)", prophet_full_fc),
    ("Seasonal-naive (single-shot)", sn_full_fc),
]:
    fixed_rows.append(
        {
            "model": name,
            "MAE (log)": eval_fixed(fc, test),
        }
    )

fixed_df = pd.DataFrame(fixed_rows)
with pd.option_context("display.float_format", "{:.4f}".format):
    print("Single-shot forecast from 31 December 2020 (130-day horizon):")
    print(fixed_df.to_string(index=False))
```

```
Single-shot forecast from 31 December 2020 (130-day horizon):
                       model  MAE (log)
        SARIMA (single-shot)     0.1439
Prophet default (single-shot)     0.1514
 Seasonal-naive (single-shot)     0.1266
```

Summary

All findings describe counter cash receipts (about 65% of revenue; see NB01), analysed on log revenue with 25 December 2019 excluded from training.

Evaluation. Rolling-origin over 122 test-window origins (1 January – 10 May 2021, entirely in the lower plateau). At each origin every model forecasts the single day 1, 7 and 30 days ahead, as in the introduction. SARIMA keeps its training-set order and re-estimates its parameters at each origin on the daily-frequency series (missing days handled by the Kalman filter; 3 of the 122 refits did not converge and kept the previous origin's estimates); every Prophet variant is refit at each origin. Baselines are computed strictly from at-origin data.

Point-estimate headline. MASE gaps between the top three models (SARIMA, Prophet default, Prophet variant C) are about 0.03–0.04 across horizons. Point estimates favour SARIMA at h = 1 (0.65) and h = 7 (0.69), and Prophet default at h = 30 (0.87, against SARIMA's 0.90). Variant C is within 0.02 of Prophet default at every horizon. Prophet variants A and B underperform Prophet default at every horizon.

Tests. One-sided Diebold–Mariano tests of each model against seasonal-naive at h = 1 and h = 7, Holm-adjusted within each horizon at 2.5%. SARIMA beats the same-weekday rule at both horizons (adjusted p = 0.0089 and 0.0011; MAE 20.7% and 17.7% lower, 95% intervals 6.9–34.4% and 8.0–27.4% lower). No Prophet variant is shown to beat it: Prophet default's 18.2% and 13.5% lower error have adjusted p = 0.039 and 0.41. SARIMA's one-sided p-values are below a Bonferroni threshold across the ten tests run (0.005) and across all fifteen model–horizon comparisons (0.0033), with a wide margin at h = 7 and a narrow one at h = 1. Under a null built from the data's own loss differentials the procedure's family-wise error rate at nominal 2.5% is 1.9% at h = 1 and 5.9% at h = 7 (`validation/dm_calibration.py`). `arch`'s SPA test, which does not studentise, does not reject on the same forecasts (p = 0.044 and 0.117); Hansen's studentised SPA does (p = 0.010 and 0.006, StepM naming SARIMA only). The reading above gives the sizes of all three procedures under the data-based null and why the Holm procedure is the test used. With about 110 forecasts, gains smaller than about 14–30% for the three leading models could not be reliably detected, depending on model and horizon.

By weekday (descriptive). At h = 7 SARIMA beats seasonal-naive on Sunday to Thursday by 20–30%, is level on Friday (1% worse) and 5% worse on Saturday. Prophet default beats it on Sunday to Thursday by 7% (Thursday) to 28% (Wednesday and Sunday), but is 16% worse on Friday and 4% worse on Saturday. Variant C, designed to help on Fridays, is worse still. The fitted-Friday-effect check shows why Prophet misses Fridays: both variants overshoot the plateau's Friday dip because their weekly patterns pool the crash and rebound (deeper Fridays) with the plateau; variant C makes it worse by *excluding* the shallower pre-COVID Fridays that were pulling default's pool towards the plateau's actual. SARIMA's weekly pattern is updated from recent weeks, so it does not overshoot.

Methodological point on look-ahead. Using NB02's full-sample break dates as Prophet changepoints would be look-ahead: the training data alone reveal only the March and May breaks, not the 4 December one. Training-only PELT here shows the correct approach and its limitation for a forecaster fixed at 31 December.

Single-shot from 31 December 2020. With every model using training data only, seasonal-naive wins at MAE 0.127, then SARIMA 0.144, then Prophet default 0.151. At a 130-day single-shot horizon, extrapolating a fitted trend hurts; the seasonal-naive rule is hard to beat.

Bottom line. For a week-ahead forecast in this test window, SARIMA's error is 17.7% lower than the same-weekday rule's, and the difference survives a one-sided test adjusted for the search across five models; a day ahead the gain is larger in point terms (20.7%) but less precisely estimated. Prophet's 13.5% lower error is not statistically established. For a single 130-day forecast from a fixed origin, the same-weekday rule is best. Both models beat the rule on every working day, Sunday to Thursday; on the local weekend Prophet loses on Friday and Saturday, while SARIMA is level on Friday and loses on Saturday.

NB03's volatility findings are not used: this notebook evaluates point forecasts only.